Project 4 Report

Sashank Bhagavatula, Anirudh Kodukula

Project Overview and Goals

For this project, we had to create the popular game of Tetris. We had to create and then implement TetrisPiece, TetrisBoard, and TetrisBrain, and then JBrainTetris, which plays the game with a brain we created. Every accessor method for the pieces and board had to run in constant time, and we made the rotations and wall kicks follow SRS.

We had four goals. First, we wanted pieces that followed SRS and returned their rotations in constant time. Second, we wanted a board that only followed legal moves according to SRS, applied SRS wall kicks, cleared rows correctly, and responded to every accessor in constant time. Third, we wanted a brain that outperformed LameBrain by a significant margin, along with a JBrainTetris that lets it play the real game. Finally, we wanted to test each layer on its own, without relying on the GUI interface.

Scope, Quality, and Limitations

Our program is meant to run a complete game of single-player Tetris, played by a human player (via JTetris) or our brain (via JBrain Tetris). All 7 pieces we implemented have all possible rotations and SRS wall kicks when the rotation is blocked. The board accepts and rejects all movement, rotation, and drop commands and clears any completed rows when the current piece settles. The brain considers positions reachable through horizontal movements and rotations before dropping the current piece and settles on the one that results in the most favorable board.

Piece and board accessors are constant time since the pieces generate all their rotations on creation, and the board maintains row/column widths, max height, and other cache info. In our simulations, the brain played about 1,250 pieces per game across 20 random games, versus around 40 for Lame Brain.

One limitation of our implementation of the game is that we haven't implemented the HOLD action.

The brain has a few downsides of its own. It doesn't look ahead to future pieces, so a placement that seems good now may turn out to be bad in the long run. In our test games, one game finished after as few as 110 pieces, while the average was around 1,250, showing that the

brain's performance varied depending on the random sequence of pieces. It also doesn't do the very best placement. Its search considers horizontal translation and rotation before dropping a piece, but not DOWN (we left that out to keep all decisions very simple), so it won't make a placement that involves moving the piece DOWN before sliding it under an overhang. In addition, it does a lot of duplicate searching since it searches again from scratch for each decision, and testMove() copies a whole board for each possible move. It is quick enough on small boards, but on larger boards it would be noticeably slow.

Algorithm and Implementation

The project is separated into four classes: the piece's implementation, the moving and placing board, the searching brain, and the GUI subclass that makes the brain play.

Breaking the project into separate classes helped us to isolate responsibilities so that we could test them individually. The Piece and Board interfaces provided an abstraction that separated the game from the graphics interface and brain. This simplified our debugging as we could more easily pinpoint where something was going wrong.

TetrisPiece

Each TetrisPiece stores its type, rotation index, bounding-box width and height, body, skirt, and references to its clockwise and counterclockwise rotations. Our implementation does not modify a piece's body or skirt after construction, and its rotation methods return piece objects that have already been computed rather than changing the existing piece. The methods getType(), getRotationIndex(), getWidth(), and getHeight() return stored values directly, while getBody() and getSkirt() return defensive copies. All of these methods still run in constant time because each piece contains only four blocks and has a bounding box no wider than four columns.

We implemented rotations using a circular doubly linked structure. The public constructor creates the piece in its initial orientation, then generates the remaining three rotations by applying the clockwise transformation (x, y) -> (y, n - 1 - x), where n is the bounding-box width. Each rotation is linked to its clockwise and counterclockwise counterparts, with the final rotation linking back to the original piece. As a result, clockwisePiece() and counterclockwisePiece() simply return stored references and run in constant time. JTetris creates the seven standard piece types and reuses their precomputed rotations throughout the game.

A shared setup() helper calculates the skirt of each rotation during construction. It initializes every column's skirt value to Integer.MAX_VALUE, then updates it to the lowest

y-coordinate of any block in that column. Columns containing no blocks retain Integer.MAX_VALUE.

The equals() method compares the type and rotation index of two TetrisPiece objects. Since these values uniquely determine a piece's body, the method doesn’t need to compare individual blocks.

If the constructor receives a null type, it prints an error message and initializes an empty piece whose rotation references point to itself, rather than throwing an exception.

TetrisBoard

The Tetris Board object represents the game board and handles moving, placing, and clearing pieces. It stores data such as a PieceType[width][height] grid, the current piece and position, the last action and how it turned out, the number of lines cleared by the last action, and the cached column heights, row widths, and max height.

All of the actions are handled by move(). For LEFT, RIGHT, and DOWN, the new position of the piece is calculated and checked using canPlace(). The piece is moved if canPlace() returns true, and SUCCESS is returned. If LEFT or RIGHT is blocked, it returns OUT_BOUNDS without moving. If DOWN is blocked, it places the piece where it is and returns PLACE. DROP moves the piece down in a loop until it can't go further, then places it and returns PLACE. NOTHING and HOLD return SUCCESS without moving the current piece or changing the grid, since we did not implement hold. If there is no current piece, every action returns NO_PIECE.

move() manages rotations as well: it just calls rotateCurrentPiece() to perform either the clockwise or counterclockwise rotation. This attempts the pre-calculated rotations of the piece, testing the current rotation index against the SRS wall-kick offsets in turn. The stick has its own wall-kick tables (because of its 4x4 bounding box), the square rotates in place with a single (0, 0) offset, and all other pieces use the standard tables. If one of these offsets works, the move succeeds; if none do, the piece's position and rotation stay the same and the method returns OUT_BOUNDS.

When a piece lands, placeCurrentPiece() copies its blocks into the grid and clears the current piece. It then clears completed rows with a bottom-to-top scan using a write pointer. Full rows are counted and skipped, while every kept row is copied down to the write pointer, and any rows left above the write pointer are emptied. This handles any number of cleared rows, adjacent or not, in O(width × height) time. Blocks in kept rows move down only by the number of cleared rows below them, so blocks can still float above empty cells, as the spec requires. Finally, it calls updateStats() to recompute the cached statistics.

nextPiece() gets a new piece onto the board. It verifies that the piece, the type, and the starting position are all non-null, then invokes canPlace() to test that the piece is placeable on the board at the start position without overlapping previously placed blocks. If the input is invalid, it simply outputs an error message and leaves the board unchanged. Otherwise, the new piece becomes current, and the start position is saved.

All accessor methods of the class execute in O(1). getCurrentPiece() and getCurrentPiecePosition() return the current game state. getCurrentPiece() returns the current piece, while getCurrentPiecePosition() returns a copy of its position. getLastAction(), getLastResult(), getRowsCleared(), getWidth(), and getHeight() return stored fields. getColumnHeight() and getRowWidth() return the pre-calculated values, or 0 if the input is out of bounds, while getMaxHeight() returns the stored maximum height. getGrid() returns the piece type at a coordinate, or null if the coordinate is out of bounds.

If we recomputed statistics every time an accessor was called, it would be expensive, so we cache them. updateStats() counts the size of each row, the height of each column, and the maximum height (tallest column), based on the filled cells. These statistics don't take into account the current piece, so we only need to update these when a piece is placed, which is the only time the grid changes.

dropHeight() returns the y-position where a piece would come to rest if it were dropped straight down with its left edge at the given x. For each column the piece occupies, it subtracts the piece's skirt value from the board's cached column height, then takes the maximum. Columns the piece doesn't occupy have a skirt of Integer.MAX_VALUE and are skipped.

testMove() makes a copy of the board and modifies that copy. It copies the array of cells, clones the cached statistics, and makes a new Point to track the location of the current piece. It copies the previous move, previous result, and rows cleared before modifying the copy. Since our gameplay methods do not modify TetrisPiece objects, both boards can use the same TetrisPiece reference, allowing the brain to simulate moves without changing the original board's gameplay state.

Lastly, equals() considers two boards equal if they have the same dimensions, the same current piece type and rotation, the same current piece position, and identical contents in every grid cell.

We represented the grid with a two-dimensional array, since blocks are always accessed by their coordinates.

TetrisBrain

In our Tetris Brain, we use the nextMove() method, which iterates through all potential positions of the current piece by moving it horizontally and rotating it, examines the resulting board state, then takes one step in the direction of the move with the highest resulting score. It is deterministic and stateless, so given the same situation, it makes the same decision each time. If there is no current piece, it returns NOTHING.

Unlike Lame Brain, which only moves blocks LEFT and RIGHT without rotation, our implementation searches for possible positions and rotations using BFS. Each search state contains a board state and the first action leading to this state. The search starts from the original board, while the subsequent states contain independent copies (cloned through testMove()). Starting from the current board, our code evaluates each position by dropping a piece with testMove() and scores the resulting board. Afterwards, it considers the LEFT, RIGHT, CLOCKWISE, and COUNTERCLOCKWISE actions, skipping any that don't give SUCCESS. We used an ArrayDeque as the BFS queue and stored each state's x, y, and rotation index in a HashSet to track visited states. Since the placed blocks are fixed, the position and rotation index of the piece serve as a key for each state.

Each discovered state keeps the first action taken on its path, which the brain eventually returns if that state produces the predicted highest-scoring placement. Since BFS explores the nearest options first, BFS tends to prefer the shortest move for multiple high-scoring options. If the best score comes from dropping at the starting position, the algorithm returns DROP. The existing move and rotate functions for the board will automatically handle wall kicks and collisions.

The brain can only return a single move at a time, not a whole sequence of moves. JBrainTetris will make another call to nextMove() on the next tick with the current state of the board as the new state, enabling it to keep testing when the piece moves into a good position. We consider each position and apply an appropriately weighted scoring function given these four features of the board:

Score = 0.8×rowsCleared - 0.5×totalHeight - 0.35×holes - 0.18×bumpiness

Of course, cleared lines are given a positive score because this is the main way to score in Tetris. The total height is calculated from the heights of each column and is used as a negative score because tall stacks are to be discouraged. Holes are penalized because they are empty cells below the top of a column, which make subsequent line clearing harder. Bumpiness is calculated by summing the height differences between neighboring columns and is penalized because a bumpier surface makes placing subsequent blocks more difficult.

The weights reflect the trade-off between clearing rows and having a flat board with fewer holes. Clearing a row directly gives a reward of 0.8, but it also greatly lowers the total height, giving an indirect reward through the height penalty. The algorithm simply chooses the

move with the highest score, without any look-ahead for future pieces. With BFS, the only moves considered are horizontal movements and rotations, so, for example, BFS will not find a placement where a piece is first moved down to reach a legal placement, and then moved horizontally or rotated.

JBrainTetris

The JBrainTetris class extends JTetris and overrides the tick() method to allow the brain to control the game. Whenever a tick occurs, whether triggered by the timer or a key press, the method ignores the original action and calls nextMove() to determine the brain's preferred move. It then passes that action to super.tick(), which handles the movement and updates the game state. All other functionality is inherited from JTetris, and the main() method launches the game.

Because the brain never returns DOWN, pieces do not move down automatically between actions, although wall kicks can sometimes change their vertical position. Once the brain selects DROP, the piece falls directly to its landing position.

Testing

We tested each part of the project separately using black-box tests that interact only with the public Piece, Board, and Brain interfaces. Because real-time gameplay can be difficult to test consistently, we created specific board states and checked the results of individual actions rather than relying only on the GUI (we did, however, play the game through the GUI to gauge how our program worked). This allowed us to test each component independently.

For TetrisPiece, we used 11 tests in BlackBoxTetrisPieceTest to check piece construction, rotation, and equality. We checked the initial bodies of the pieces and all four rotations of the T, stick, and left dog pieces. We also tested whether four clockwise rotations returned a piece to its original orientation, whether one counterclockwise rotation produced the same result as three clockwise rotations, and whether rotations correctly wrapped between indices 0 and 3. The other tests checked bounding-box dimensions, skirt calculations for columns without blocks, and equality between pieces. These tests were useful because incorrect rotations or skirts could cause problems with movement and placement later on in the game.

For TetrisBoard, we made tests to verify movement, placement, row clearing, and board state. We tested empty boards, actions taken without a current piece, and invalid inputs to nextPiece(). We also checked movement against walls and existing blocks, including whether unsuccessful horizontal movements left the piece unchanged and whether a failed DOWN action placed the piece. For DROP, we verified the landing position and the updated column heights, row widths, and maximum height. We also tested dropHeight() with different board configurations, including overhangs, to verify that it correctly accounted for the piece's skirt.

Row clearing was tested with single completed rows, multiple adjacent rows, non-adjacent completed rows, and four-row clears. We checked that completed rows disappeared, remaining blocks shifted downward correctly, and the board's statistics were updated. For rotations, we tested changes in rotation index, wall kicks against board boundaries and placed blocks, the stick piece's separate wall-kick rules (it has a 4x4 bounding box instead of a 3x3 one), and rotations that should fail because no valid position exists. We also verified that testMove() produced an independent board without changing the original and that equals() correctly identified equivalent board states. These tests covered both common gameplay actions and edge cases that would be difficult to reproduce consistently through manual play.

For TetrisBrain, rather than checking the exact scores produced by our brain algorithm, we focused on the brain's observable behavior. We checked that nextMove() did not modify the input board and that identical board states produced the same action. We also tested different piece types and rotations near walls and at different heights to see whether the brain could find placements without creating unnecessary holes. Other scenarios included filling gaps on either side of the board, rotating a stick piece into a narrow opening to clear four rows, and avoiding placements that created difficult overhangs.

We also used longer simulations to evaluate the brain's performance across 25 random games. These tests checked whether the brain selected legal actions, preserved the expected number of blocks after placement and row clearing, and could survive an average of at least 100 pieces per game. Testing multiple random piece sequences was useful because a brain that performs well on one board configuration may struggle with different combinations of pieces. These simulations provided a broader measure of the brain's effectiveness than individual placement tests alone.

Most board and brain tests constructed specific scenarios by adding pieces at known positions with nextPiece() and placing them with DROP. For the brain tests, we used a placeFixture helper that aligned pieces based on their actual block coordinates rather than the origin of their bounding boxes. This allowed us to create predictable board configurations without being affected by the differences between each piece’s bounding box and its actual block positions.

We also calculated expected results manually for cases where the exact outcome mattered. For example, when testing wall kicks, we determined which offsets should fail and which should produce a successful rotation, then compared the expected position against the board's actual result. This was especially important for the stick piece, which uses different wall-kick offsets from the other pieces. By testing specific expected outcomes rather than only checking whether an action succeeded, we could identify incorrect movements that might be accepted in the program but violate SRS rules.

Separating the project into pieces, board logic, and brain made it easier to test each component independently without relying on the graphical interface. We found that creating

specific board states and checking expected results was much more reliable than trying to reproduce unusual situations during normal gameplay. In particular, manually checking rotations, wall kicks, and row clearing helped us look at cases where a small error could affect the game later on.

Testing that testMove() and nextMove() did not change their input boards was also important because the brain evaluates many possible moves using copies of the current state. If these operations changed the original board, the brain could interfere with the actual game. Finally, testing the brain across different random piece sequences provided a more useful evaluation of its overall performance than looking at a single game.

Interesting Results

We tested the implementation of Tetris Brain on 10×24 boards with pieces spawning at y = 20 and ending the game when the stack went over height 20. We compared our brain to that of Lame Brain using the same 20 random seeds. Our brain averaged 1,250 pieces per game, compared to 39 for Lame Brain. Our brain's shortest game was 110 pieces, and its longest was 2,600. Meanwhile, LameBrain placed between 33 and 45 pieces. This difference was due to our algorithm's added consideration of rotations, board height, holes, and surface unevenness rather than just height.

Although the mean result of the brain was quite decent, results varied a lot from one set of pieces to the other. As noted earlier, the shortest game was 110 pieces and the longest 2,600. The shortest one always appeared with the same random seed, meaning that there are simply some sequences of pieces that the algorithm underperforms in. Our brain simply checks the current piece and has no idea what the next one will be, so it cannot plan for these sequences.

Finally, we discovered the brain could make decisions quickly even with so many placements to consider. Each piece required, on average, three moves and a drop. This demonstrated to us that BFS was feasible for the Tetris board whilst still enabling the brain to consider rotations and wall kicks.

Assumptions

The project follows the coordinate system where (0, 0) represents the bottom-left corner of the board, x increases to the right, and y increases upward. A piece's position refers to the bottom-left corner of its bounding box, which may extend outside the grid as long as all of its blocks remain within the board. Our code uses the seven standard tetrominoes with their provided spawn bodies and square bounding boxes, allowing us to apply the same coordinate transformation to rotate every piece. The provided SRS wall-kick tables are treated as correct and indexed by the source rotation, while the square piece rotates in place without wall kicks.

We've also assumed that the grid is only modified when a piece is placed, since moving or rotating the current piece does not affect previously placed blocks. This allows cached statistics, including column heights, row widths, and maximum height, to be recalculated only after placement.

The HOLD action is treated as equivalent to NOTHING, and the brain also operates using only the current piece and board state, without information about upcoming pieces. Because nextMove() returns only one action and is called once per tick, the brain reevaluates the board after every movement rather than committing to a certain sequence of actions.

Problems Encountered

Implementing TetrisPiece was tricky because of constant-time rotations and the links between rotation states. Since clockwisePiece() and counterclockwisePiece() had to run in constant time, we were not allowed to compute the rotated shape each time it was requested. We computed all four in the constructor and linked them in a circular doubly linked list. We also had to check whether the last rotation was connected to the original one. Otherwise, after some number of rotations, we would end up referencing null. During implementation, we encountered compilation problems because we changed the rotation field name to something else, while setup() still mentioned the previous field. We corrected the reference of the rotation field so that each piece can remember the rotation index.

Another problem was drawing the various shapes correctly in their given SRS bounding boxes. In particular, we used the coordinate transformation (x, y) → (y, n - 1 - x) to rotate blocks in a fixed square box. For the stick, we handled its 4x4 bounding box using the same coordinate transformation with n = 4 to simulate how blocks move inside that box. For the square, we used four rotation objects with indices 0–3, even though the shape stays the same after rotation. We linked them all up so that a rotation action would properly change the index but the blocks wouldn't move at all. Computing skirts had to handle empty columns by initializing each entry to Integer.MAX_VALUE and replacing it when a smaller value was found.

When implementing Tetris Board, we ran into a number of bugs related to movement, collision, and the related methods. One bug we found early was that a move changed the piece's shape, because we edited its body array directly instead of only updating its position. We fixed this by leaving the body unchanged and only updating the position once the move was confirmed legal. We also used to check for collisions incorrectly, because the old check used coordinates relative to the piece rather than relative to the board. We fixed this by adding the origin of the piece to the relative coordinates for each block, and then checking those new values to see if they're outside the board's bounds or if the position is currently filled, via the helper canPlace().

The second problem was height calculations and when to drop pieces. Our first height

calculations counted the number of filled cells in the column, which did not work if there were holes in the columns. We fixed this by taking the highest occupied y-value and then adding one so that the height was based on the max stack rather than the number of blocks. We also had to make sure that when a DOWN action was blocked, the piece was placed rather than rejected. We fixed this by copying the piece into the grid, removing completed lines and dropping a piece once DOWN was no longer possible.

There was another difficulty with clearing rows, as there can be multiple rows cleared at the same time. If we just remove the full rows, we move the other blocks down, but we do not correctly compact the incomplete rows to fill the previous empty spaces. We fixed this by using separate reading and writing positions to scan the board from bottom to top, skipping completed rows and moving the rest down into the vacant space.

We also had issues with the dropHeight() method. Our initial implementation did not take into account the shape of the piece or the height of existing blocks properly. We were able to correct this by using the board column heights and the piece's skirt to locate the final position of the piece. Furthermore, our initial implementation would not allow the piece to land in positions where the bounding-box origin would go negative, even though the blocks would be falling within the board. We got around this by removing that restriction, which allowed us to land pieces with any empty rows at the bottom of the bounding box. We also encountered an issue where rotating the square piece repeatedly would cause an error because the wall-kick table only had one entry for each rotation. We were able to fix this by adding a zero-offset entry for each of the 4 rotation indices.

Tetris Brain also had its own problems. Tetris Brain has to consider all possible combinations of horizontal shifts and rotations without testing a state twice. We solved this by using BFS with a HashSet to track visited states based on piece position and rotation index. Another issue was how to return a single move when the optimal piece placement might actually be achieved after multiple moves. Our solution was to store the first move along the path of searching for the optimal move and return that move.

Finally, the brain had to consider possible placements without affecting the game state. We utilized testMove() to simulate moves on separate boards; this enabled us to effectively evaluate placements based on cleared lines, height, holes, and bumpiness. This isolates the brain evaluation from the current game state, allowing us to try different placements.

Social Impact

There are some accessibility considerations that we need to take into account with our implementation of Tetris, which may make it hard for visually impaired or physically disabled players to enjoy the game. Our game makes extensive use of colors, keyboard controls, and reflexes.

Another major design issue is playability for those with color vision deficiencies. The game differentiates the seven tetrominoes by having all of them be different colors, including the red and green polygons, which make up the left and right dog pieces. These pieces could easily be mistaken by people who have any form of red-green color blindness. According to the National Eye Institute, 1 in 12 men has some form of color vision deficiency. While the shape of the pieces also differentiates them, it would be more playable if the pieces had patterns or letters, or if there was a colorblind mode. Additionally, larger blocks, more contrast, and stronger outlines would be more easily visible for those with low vision. The game can benefit from sounds or screen-reader functions to inform the player about the current game state, such as the current piece and position or the height of the columns.

Another accessibility issue is our reliance on keyboard controls and timing requirements. Our implementation assigns fixed keys for moving and rotating pieces around the board, but players with limited motor control could find it difficult to perform those actions. We could add support for remapping controls or help players by allowing switches and adaptive controllers, like Microsoft's Xbox Adaptive Controller (designed to support players with limited hand and arm mobility). We have a sliding speed scale, but a pause button would support players by giving them more time to think about the next move. The Tetris Brain could be adapted into an aid that pauses the game and assists players with keyboard input.

When designing software, accessibility is taken into account since software should adapt to the user instead of the user adapting to the software. The World Wide Web Consortium's Web Content Accessibility Guidelines (WCAG) serve as a good example of accessibility. The WCAG guidelines recommend that one should not communicate information with color alone. These guidelines obviously relate specifically to content on the web, but the concept can be applied to our game. Designing for accessibility at the beginning minimizes changes down the line, and it applies to all players, including players with disabilities. Alternative controls, speeds, and visual cues are likely to make things easier not just for those with disabilities but also for any players with different ways they like to use a game.

References

National Eye Institute. “Color Blindness.”
https://www.nei.nih.gov/eye-health-information/eye-conditions-and-diseases/color-blindness
Microsoft. “Xbox Adaptive Controller.”
https://www.xbox.com/en-US/accessories/controllers/xbox-adaptive-controller
World Wide Web Consortium (W3C). “Use of Color.” Web Content Accessibility Guidelines (WCAG) 2.2, Understanding documents.
https://www.w3.org/WAI/WCAG22/Understanding/use-of-color.html

Log

Phase	Driving Together (hrs)	Working Separately (hrs)	Working Separately: Anirudh (hrs)	Working Separately: Sashank (hrs)	Notes
Design	1	1	0.5	0.5	We reviewed the specifications and planned the class structures.
Implementation: TetrisPiece	2	2	1	1	We implemented rotations, linked references, skirts, and equality.
Implementation: TetrisBoard	3	4	3	1	We implemented movement, collision detection, wall kicks, row clearing, and board statistics.
Implementation: Brain + JBrainTetris	2	3	1	2	We developed BFS placement search, the scoring function, and GUI integration.
Testing	1	4	2	2	We developed JUnit tests for

					pieces, board operations, and brain behavior.
Debugging	2	2	0.5	1.5	We fixed issues with movement, drop heights, rotations, row clearing, and brain decisions.
Report	1	5	2	3	We documented the implementation, testing, results, and challenges.
Total	**12**	**21**	**10**	**11**	**33 session-hours (45 person-hours)**